FLIGHT SOFTWARE V1 — DEVELOPMENT ROADMAP — REVISION 5

One Bus, One Sitting at a Time: I2C-Only Comms, FPGA Compression, and a 3-Hour Task Plan

> *"We didn't make a motor spin. We made the logic flow." — the guiding principle for every phase before real subsystem logic.*

Status: living document, revision 5 — I2C is now the only inter-MCU CSP bus (CAN removed), SPI is the local sensor bus, UART is debug-only; the SatNOGS-COMMS FPGA is now a compression accelerator, not neuromorphic compute; a new BrainChip Akida1500 unit owns SNN work; Phases 1–6 rewritten as ~3-hour task tables

Scope: ADCS / EPS / Thermals / Camera PCB as separate MCUs; OBC + Comms + FPGA as three units on the SatNOGS-COMMS board; Akida1500 as a fourth, standalone, OBC-hosted accelerator

Companion documents: `docs/system_report.pdf`, revisions 1–4 of this roadmap

§ Contents

§ Guiding Principle

- **Foundation before features.** Every phase before Phase 5 produces infrastructure and conventions every subsystem reuses — not subsystem-specific behavior.
- **Generalize the second time, not the first.** ADCS was built once, by hand. Phase 1 turns that into something EPS, Thermals, and Camera can each pick up without re-deriving it.
- **Parallel-safe by construction.** Every Phase 2+ workstream should be doable by a different person without stepping on anyone else's files.
- **Simplicity over separation.** One real/sim split, not two; fewer top-level directories; the same "swap one file to change hardware" guarantee with less structure to maintain.
- **One sitting, one task** REV 5 **.** Every item in Phases 1–6 is sized to be finishable in about three hours. Depends-on tells you what has to exist first; done-when tells you when to stop and check the next item off, not keep polishing.

§ Architecture Decisions (Now Settled)

TOPOLOGY SETTLED

EPS, ADCS, Thermals, and Camera are each their own physical MCU board. The SatNOGS-COMMS board contributes three more units (OBC, Comms, FPGA); a fourth, standalone accelerator (Akida1500) is hosted by OBC but isn't part of the SatNOGS-COMMS board itself.

OBC, Comms, FPGA, and Akida1500

Unit	Silicon	OS / Toolchain	Role	Directory
OBC	Zynq-7000 PS (dual Cortex-A9)	Linux (PetaLinux/Yocto)	Mission computing, CSP hub, subsystem command/telemetry	`apps/obc/`
Comms	STM32H743	Zephyr + libsatnogs-comms, as shipped	TMTC, dual-band UHF/S-band radio control, ground link	`apps/comms/`
FPGA	Zynq-7000 PL fabric	none — synthesized gateware	**Compression** — shrinks telemetry, command, and sensor data before transmission	`fpga/`
Akida1500	BrainChip AKD1500 (separate chip/module)	MetaTF / Akida SDK	Neuromorphic / SNN compute — purpose TBD	`akida/`

FPGA'S JOB CHANGED: COMPRESSION, NOT NEUROMORPHIC COMPUTE

CHANGED IN REV 5

Revision 4 put spiking-neural-network compute on the PL fabric. That workload moves to its own dedicated chip (Akida1500, below); the FPGA instead becomes a **downlink compression accelerator** — it sits between OBC's CSP/telemetry-aggregation layer and the hand-off to Comms, shrinking payloads (telemetry, command-related data such as ACK/logs, and sensor/imagery data) before they go out over RF. Compression algorithms, ratios, and latency budgets are things a requirements pass can actually pin down — a much better-scoped problem than "figure out what an SNN does here." Still reached the same way as before: the on-chip AXI interconnect from OBC's Linux side, not the external I2C bus, and not a CSP address.

DECOMPRESSION IS A GROUND-SEGMENT CONCERN

The FPGA only needs to implement the compress path — nothing downstream of transmission runs on the satellite. Whoever owns ground-station tooling needs to know the chosen algorithm/format so they can decompress on receipt; track that coordination alongside Phase 4A.

Akida1500 is new: a dedicated neuromorphic chip, not a fabric workload. Separate silicon from the SatNOGS-COMMS board, its own top-level directory because its toolchain is a third kind entirely — BrainChip's MetaTF/Akida SDK (Python-based model conversion and deployment), not CMake/C and not Vivado/HDL. Recommended treatment mirrors the FPGA: hosted by OBC as a local accelerator, invoked synchronously for inference, not a CSP node. Its physical interface to OBC (PCIe, SPI, or otherwise) isn't confirmed yet — see Fork C.

OBC ↔ Comms is a normal CSP link. Comms sits on the same internal I2C bus as every other subsystem, at its own CSP address — ground-bound traffic is a command OBC hands to Comms (after FPGA compression); Comms puts it on the RF link.

Net effect on the node graph: six physical CSP nodes — OBC, Comms, ADCS, EPS, Thermals, Camera. FPGA and Akida1500 are not CSP nodes.

> **BUS MEDIUM: I2C FOR INTER-MCU CSP, SPI FOR LOCAL SENSORS, UART FOR DEBUG ONLY** SIMPLIFIED IN REV 5

Bus	Role	libcsp support
I2C	The only inter-MCU CSP bus	`csp_if_i2c.c` against three backends: Linux (SIM everywhere + real OBC), Zephyr (real Comms, libcsp's own arch port), and a to-be-built FreeRTOS/STM32 HAL backend (real ADCS/EPS/Thermals/Camera, Phase 6).
SPI	Local sensor bus, per board — IMU, magnetometer, thermistor, sun sensor, GPS, camera	Not a CSP interface — point-to-point per sensor, local to each MCU, never between boards.
UART	Debug console only	Standard serial everywhere. No GPS role — GPS is now SPI-attached (see Fork B).
~~CAN~~	Removed entirely — one inter-MCU bus, not two, means one less axis to build, test, and choose between.	

This retires the `COMMS_BUS` CMake option from revision 4 — there's no longer a second bus medium to pick between. One less build flag, one less thing every subsystem's Definition of Done needs to verify twice.

§ Directory & Driver Convention

Same simplified rule as before — `shared/interfaces/` for every abstract contract, `platform/{real,sim}/drivers/` for every concrete implementation — now with `comms_can.c` dropped and a new `akida/` sibling next to `fpga/`.

```
shared/
├── csp/                      # unchanged -- protocol layer
└── interfaces/                # every abstract contract, bus + peripheral together
    ├── comms_bus.h            # I2C-only now -- no medium selection left to abstract over
    ├── imu.h                  # SPI-attached
    ├── magnetometer.h         # SPI-attached
    ├── thermistor.h           # SPI-attached
    ├── gps.h                  # SPI-attached -- moved off UART, see Fork B
    └── ...                    # one header per interface, added as needed

platform/
├── real/
│   └── drivers/
│       ├── comms_i2c.c        # was comms_can.c + comms_i2c.c -- now just this one
│       ├── imu_<part>.c
│       └── ...
└── sim/
    └── drivers/
        ├── comms_i2c.c        # same backend serves real OBC too
        ├── imu_mock.c         # "logic not motor" -- fake values, logs every call
        └── ...

apps/
├── obc/                       # Linux, real and SIM both
├── comms/                     # Zephyr + libsatnogs-comms, own build system (West)
├── adcs/                      # done -- FreeRTOS, reference implementation
├── eps/                       # FreeRTOS, Phase 2
├── thermals/                  # FreeRTOS, Phase 2
└── camera/                    # FreeRTOS, Phase 2

fpga/                          # Zynq PL gateware -- compression accelerator
└── ...                        # Vivado/HDL project -- own toolchain, not CMake/C

akida/                         # NEW -- BrainChip Akida1500, neuromorphic/SNN accelerator
└── ...                        # MetaTF/Akida Python SDK -- a third toolchain
```

Notes on this convention

- **`comms_bus.h` collapses to one real implementation.** With CAN gone, there's exactly one `comms_i2c.c` pair (real/sim) — `HW_MODE` alone decides which compiles.
- **Start flat inside `drivers/`, don't pre-organize by subsystem.** No cross-subsystem reuse pressure yet.
- **`apps/comms/`, `fpga/`, and `akida/` all sit outside the CMake build.** Three different toolchains (West/Zephyr, Vivado/HDL, MetaTF/Python), three siblings in the tree for discoverability.

§ Revised CSP Address & Port Table

Unchanged from revision 4 — I2C replaces CAN as the physical carrier, but CSP addresses and ports don't care what's underneath them.

Node	CSP Addr	Cmd Port	Telem Port	Status
OBC (Zynq PS, Linux)	1	—	—	DONE (SIM); real HW = Phase 6
ADCS	2	10	20	DONE (position cmd)
EPS	3	11	21	RESERVED
THERMALS	4	12	22	RESERVED
CAMERA PCB	5	13	23	RESERVED
COMMS (STM32H743, Zephyr)	6	14	24	RESERVED
Ground (via Comms)	—	—	30	PHASE 3

§ Remaining Design Forks

FORK A — COMMAND RELIABILITY MODEL DECIDE PER-COMMAND IN PHASE 5

CSP's RDP mode is enabled but unused; default to best-effort for high-rate telemetry, require RDP or explicit ACK/NACK for anything that changes subsystem state. More relevant now that Comms is a real radio link — RF is lossier than I2C.

FORK B — GPS OWNERSHIP DECIDE WHEN GPS IS ACTUALLY ADDED

GPS moved off UART this revision — it's now an SPI-attached sensor like the IMU or magnetometer. Open question unchanged: which board owns it — ADCS (orbit/attitude determination) or OBC (ground-contact scheduling).

FORK C — AKIDA1500 PHYSICAL INTERFACE & HOST NEW IN REV 5

DECIDE BEFORE PHASE 4B.2

BrainChip Akida parts are typically PCIe- or SPI-attached to a host processor; which one this project's module uses, and whether the Zynq PS (OBC) has the right physical interface exposed, isn't confirmed. Hardware research, not a design preference — resolve by reading the chosen module's datasheet.

§ How to Read Phases 1–6

Every phase below is a table: **#** (task ID) · **Task** · **Depends on** · **Est.** (rough time, ~3h target) · **Done when** (the concrete signal you're finished, not a feeling). Work through a phase top to bottom unless a dependency says otherwise — most tasks only depend on the row directly above them. Estimates are honest guesses, not commitments; bench/hardware items in Phase 6 are the least predictable and may take more than one sitting.

1 Foundation & Reusable Infrastructure

Depends on: nothing (topology decided)

~17 tasks, ~40 hours total

1A. Directory & bus convention migration

#	Task	Depends on	Est.	Done when
1.1	Create `shared/interfaces/`; move `platform/include/v_bus.h` → `shared/interfaces/comms_bus.h`	—	1.5h	File moved, includes updated, project still builds
1.2	Create `platform/{real,sim}/drivers/`; move `v_bus.c` / `spi_driver.c` → single `comms_i2c.c` each (no CAN)	1.1	2h	Both files moved and renamed, `HW_MODE` still selects correctly
1.3	Update `csp_network.c`'s include and any remaining v_bus/SPI-named references project-wide	1.2	1h	`grep -r v_bus` returns nothing outside comments/history
1.4	Remove the `COMMS_BUS` concept if any placeholder exists; confirm `HW_MODE` is the only build-time bus switch left	1.3	0.5h	Clean cmake configure shows no dangling `COMMS_BUS` option

1.5	Full clean rebuild + rerun existing `v_bus_test` / `position_command_test`	1.4	1h	Both existing tests still pass unmodified

1B. I2C SIM backend (replaces the old SPI/v_bus SIM path)

#	Task	Depends on	Est.	Done when
1.6	Design the I2C SIM transport (Unix-socket-based, same shape as the old v_bus SIM, adapted for I2C addressing)	1.5	2h	Design fits in a short doc comment atop `comms_i2c.c`, no code yet
1.7	Implement `platform/sim/drivers/comms_i2c.c`	1.6	3h	Compiles, a trivial two-process send/receive test passes
1.8	Implement `csp_if_i2c.c`'s SIM-side integration (mirrors what `csp_if_spi.c` did)	1.7	3h	`csp_network_init()` works unchanged from the caller's point of view
1.9	Port ADCS + OBC's existing position-command flow onto the new I2C SIM backend	1.8	2h	`apps/obc` and `apps/adcs` build and link against `comms_i2c.c`
1.10	Regression check: rerun `position_command_test` end-to-end over I2C SIM	1.9	1.5h	Test passes with the same pass criteria as before (repeated cycles, no hang)

1C. Shared command/telemetry envelope

#	Task	Depends on	Est.	Done when
1.11	Design the shared envelope (command ID, sequence number, standard ACK/NACK reply)	—	2h	Short design doc or header comment describing the struct/enum layout
1.12	Extend `csp_commands.h` with the envelope types; apply to the existing ADCS position command	1.11, 1.10	2.5h	ADCS's existing command still works, now carrying the envelope fields

1D. Peripheral interface pattern (SPI sensors)

#	Task	Depends on	Est.	Done when
1.13	Write `shared/interfaces/imu.h` — first SPI-attached peripheral contract	—	1.5h	Header compiles standalone, documents the SPI assumption explicitly
1.14	Write `platform/sim/drivers/imu_mock.c` — believable fake values, logs every call	1.13	2h	A trivial test program links it and gets plausible-looking values

1.15	Write `platform/real/drivers/imu_<part>.c` — honest stub (clear "not implemented yet" error, not silent garbage)	1.13	1.5h	`HW_MODE=ON` build compiles and the stub's error path is exercised once
1.16	Repeat the mock+stub pattern for `magnetometer.h` and `thermistor.h`	1.14, 1.15	3h	Both interfaces have mock + stub pairs, same shape as the IMU
1.17	Write `shared/interfaces/gps.h` as SPI-attached (not UART) per Fork B, with a mock backend	1.16	2h	GPS mock exists; no UART code references GPS anywhere

1E. Scaffolding template, test harness, CI

#	Task	Depends on	Est.	Done when
1.18	Extract `apps/adcs`'s structure into a copyable template	1.10	3h	Running the template produces a buildable, empty skeleton subsystem
1.19	Write `directory_conventions.md` capturing the full convention from this document	1.18	2h	Doc committed, cross-referenced from this roadmap
1.20	Extract `test_position_command.c`'s spawn/capture/assert pattern into a reusable test-harness support file	1.10	3h	A second, trivial test can reuse it without copy-pasting the spawn logic
1.21	Wire `ctest` into CI so it runs on every push/PR	1.20	2h	A CI run is visibly green on the current branch

1F. Foundation Definition of Done

#	Task	Depends on	Est.	Done when
1.22	Dry run: stand up one throwaway scaffolded subsystem using **only** the template + docs, from zero to a passing sabotage-verified integration test	1.18–1.21	3h	This *is* the Foundation DoD — if it takes longer than an afternoon or needs undocumented tribal knowledge, Phase 1 isn't actually done

2 Parallel Subsystem Scaffolding

Depends on: **Phase 1 complete** — four owners, fully parallel with each other and Phase 3

~15 tasks per CMake track × 3 (EPS, Thermals, Camera) + one Zephyr-flavored track for Comms

2A. Canonical checklist — repeat once each for EPS, Thermals, Camera

#	Task	Depends on	Est.	Done when

2.1	Scaffold `apps/<subsystem>/` from the Phase 1.18 template	Phase 1	2h	Builds cleanly in `HW_MODE=OFF`, does nothing yet
2.2	Wire FreeRTOS scheduler + static allocation hooks (mirrors `freertos_hooks.c`)	2.1	2h	Scheduler starts, idle task runs, no crash
2.3	Implement `command_handler` — CSP bind/listen/accept/read loop on the subsystem's cmd port	2.2	3h	A hand-sent CSP packet is received and logged
2.4	Implement task stubs with non-blocking notify-wait activation logging, no real logic	2.3	3h	Sending a command visibly activates the right task(s) in logs
2.5	Implement the telemetry task + reply-to-OBC send path on the telem port	2.4	2h	OBC receives a telemetry reply after sending a command
2.6	Wire onto the I2C SIM bus alongside OBC/ADCS; manual multi-process run	2.5	2h	A real (non-test) run shows repeated command/telemetry cycles, not just one
2.7	Write the sabotage-verified integration test, adapted from `test_position_command.c`	2.6	3h	Test passes; deliberately breaking the command path makes it fail correctly
2.8	CI registration + Phase 2 DoD pass	2.7	1h	Test runs in CI; DoD checklist (Appendix) checked off

~18h per subsystem, ×3 = ~54h across EPS/Thermals/Camera, doable in parallel by three different owners.

2B. Comms (Zephyr) — same shape, different toolchain

#	Task	Depends on	Est.	Done when
2C.1	Set up `apps/comms/` as a West/Zephyr workspace targeting STM32H743; pull in libsatnogs-comms as a module	Phase 1 (wire contract only)	3h	`west build` succeeds against a stock Zephyr target
2C.2	Wire libcsp's Zephyr arch port + `csp_if_i2c.c` against Zephyr's native I2C driver/devicetree binding	2C.1	3h	CSP init succeeds on a Zephyr build (real or emulated target)
2C.3	Implement Comms' CSP address/ports (6 / 14 / 24) — command handler equivalent on Zephyr's thread/message-queue model	2C.2	3h	A hand-sent CSP packet from OBC's SIM is received and logged by Comms
2C.4	Stub the ground-relay path: log "would relay to RF" without touching real libsatnogs-comms TX yet	2C.3	2h	Relay stub is exercised by a manual command from OBC
2C.5	Telemetry-equivalent thread reporting Comms' own health back to OBC	2C.4	2h	OBC receives a telemetry reply from Comms

2C.6	Set up a SIM/emulation story for Comms (Zephyr native emulation or QEMU) testable without real hardware	2C.5	3h	The full command/telemetry cycle runs on a Mac without STM32 hardware attached
2C.7	Sabotage-verified integration test for Comms' scaffolding milestone	2C.6	3h	Test passes; deliberately breaking the relay path makes it fail correctly

Definition of Done (per subsystem, all four tracks)

1. Builds cleanly in its own toolchain.
2. One command flows end-to-end (OBC → CSP → I2C → command handler → task/thread → fan-out).
3. At least one task/thread sends telemetry back.
4. Uses Phase 1D's mock driver pattern for anything hardware-facing (CMake boards); Comms uses Zephyr's own native/emulated I2C for the equivalent SIM story.
5. Sabotage-verified integration test.
6. CSP address/ports match the table above.

3 OBC Services (CI, TTS, LC, TO)

Depends on: **Phase 1** + at least one live subsystem — can start in parallel with Phase 2

~11 tasks, ~28 hours

#	Task	Depends on	Est.	Done when
3.1	CI (Command Ingest) — spec: envelope shape from ground vs. local source	Phase 1	2h	Short spec doc, no code
3.2	CI — skeleton + decode of the Phase 1.12 envelope	3.1	2h	Decodes a hand-crafted test envelope correctly
3.3	CI — dispatch decoded commands to the correct subsystem's CSP address/port	3.2	3h	A CI-issued command reaches ADCS (or any live subsystem) end-to-end
3.4	TTS (Time Tag Scheduler) — spec: what "scheduled command" means (absolute vs. relative)	Phase 1	2h	Short spec doc, no code
3.5	TTS — skeleton + scheduling data structure (sorted queue or similar)	3.4	2h	Can enqueue/dequeue a scheduled command in unit-test isolation
3.6	TTS — execution trigger wired to CI's dispatch path	3.5, 3.3	3h	A command scheduled 5s out actually fires through CI at the right time
3.7	LC (Limit Checking) — spec + threshold table format for at least one telemetry field	Phase 1	2h	Short spec doc, no code

3.8	LC — monitoring loop reading live telemetry against thresholds	3.7, live Phase 2 subsystem	3h	Deliberately out-of-range telemetry triggers a logged limit violation
3.9	TO (Telemetry Output) — spec + skeleton: aggregation into a downlink-shaped structure	Phase 1	2h	Short spec doc + compiling skeleton
3.10	TO — aggregation logic pulling from at least one live subsystem's telemetry	3.9	3h	Aggregated structure visibly contains real telemetry values
3.11	Design + document the OBC↔Comms ground-relay port/payload shape	2C.4 or later	2h	Decision written down, not left implicit
3.12	Implement the relay: TO hands aggregated telemetry to Comms per 3.11; Comms logs "would transmit"	3.10, 3.11, 2C.4	3h	A full OBC→Comms relay is observable in logs on a manual run
3.13	Integration test: full uplink (CI) / downlink (TO→Comms) round trip, sabotage-verified	3.3, 3.12	3h	Test passes; breaking either direction fails it correctly

4 FPGA Compression & Akida1500 Neuromorphic

Depends on: nothing above for either sub-track — both fully parallel from day one, each a different discipline from embedded C/RTOS work

~10 tasks, ~28 hours across two independent tracks with two different owners

4A. FPGA compression accelerator

#	Task	Depends on	Est.	Done when
4A.1	Requirements pass: which payload types need compression, target ratio/latency per type, lossless vs. lossy	—	3h	Written requirements doc, no HDL yet
4A.2	Prototype the chosen compressor **in software first** (e.g. reference LZ4/DEFLATE) to validate the algorithm before HDL	4A.1	3h	Software prototype compresses a real sample payload at the target ratio
4A.3	HDL: implement or integrate the compression core in Vivado targeting the PL fabric	4A.2	3h (likely repeats)	Core simulates correctly in Vivado against the same sample payload
4A.4	AXI integration: PS-PL driver from OBC's Linux side (UIO/kernel driver) to feed data in and read output back	4A.3	3h	A Linux userspace test program round-trips a payload through the FPGA
4A.5	Wire into OBC's downlink path: Phase 3's TO routes payloads through the FPGA before Comms hand-off	4A.4, 3.12	3h	A real telemetry payload is visibly compressed before the Comms hand-off

4A.6	End-to-end validation: compress → (stub) transmit → decompress in a ground-side test tool → compare	4A.5	3h	Decompressed output matches the original byte-for-byte (lossless) or within tolerance (lossy)

4B. Akida1500 neuromorphic accelerator

#	Task	Depends on	Est.	Done when
4B.1	Requirements pass: what problem the SNN solves, what data it needs, what "done" looks like for a first integration	—	3h	Written requirements doc, no code yet
4B.2	Confirm the physical interface (PCIe/SPI/other) and host once a module is chosen — resolves Fork C	4B.1	2h	Interface confirmed against the module's actual datasheet, written down
4B.3	Stand up BrainChip's MetaTF/Akida SDK dev environment; run a stock example model	4B.2	3h	A vendor example model runs and produces expected output
4B.4	First integration milestone per 4B.1's definition (e.g. classify a fixed test vector, report to OBC)	4B.3	3h	The milestone from 4B.1 is demonstrably met, not just "the SDK works"

5 Real Subsystem Logic, Sensors & Actuators

Depends on: that subsystem's own Phase 2 (ADCS's Phase 5 can start now). Comms' equivalent is mostly vendor-provided already (Phase 2B); FPGA/Akida's are Phase 4.

~17 tasks, ~45 hours across four independent tracks

ADCS

#	Task	Depends on	Est.	Done when
5.1	Real IMU driver against Phase 1D's SPI interface	ADCS Phase 2	3h	Real sensor values read over SPI, sane ranges
5.2	Real magnetometer + sun-sensor drivers	5.1	3h	Both produce sane values over SPI
5.3	B-dot detumble control law	5.2	3h	Detumble logic runs against real/logged input, produces plausible actuator commands
5.4	Sun-pointing control law	5.3	3h	Same, for the sun-pointing mode
5.5	Kalman filter attitude estimation	5.2	3h (multi-session)	Estimate tracks a synthetic/logged trajectory within tolerance

5.6	Mode state machine wiring (`adcs_manager.h`) tying 5.3–5.5 together	5.3, 5.4, 5.5	3h	Mode transitions happen correctly under test scenarios
5.7	Fault handling + heartbeat watchdog	5.6	2h	A forced fault triggers the correct safe response

EPS

#	Task	Depends on	Est.	Done when
5.8	Real battery/solar telemetry drivers	EPS Phase 2	3h	Real values read, sane ranges
5.9	Power-rail switching logic	5.8	3h	Commanded rail state changes reflected in telemetry
5.10	Low-battery → safe-mode signal feeding Phase 3's LC	5.9	2h	Forced low-battery condition triggers the expected LC-visible signal
5.11	Fault handling + heartbeat watchdog	5.10	2h	A forced fault triggers the correct safe response

Thermals

#	Task	Depends on	Est.	Done when
5.12	Real thermistor SPI driver	Thermals Phase 2	2h	Real values read, sane ranges
5.13	Heater control logic	5.12	2h	Commanded heater state changes reflected correctly
5.14	Fault handling + heartbeat watchdog	5.13	2h	A forced fault triggers the correct safe response

Camera PCB

#	Task	Depends on	Est.	Done when
5.15	Capture/readout driver over SPI	Camera Phase 2	3h	A real frame is captured and readable
5.16	SPI bulk-link implementation for getting imagery off-board	5.15	3h	A captured frame is transferred off-board intact
5.17	Fault handling + heartbeat watchdog	5.16	2h	A forced fault triggers the correct safe response

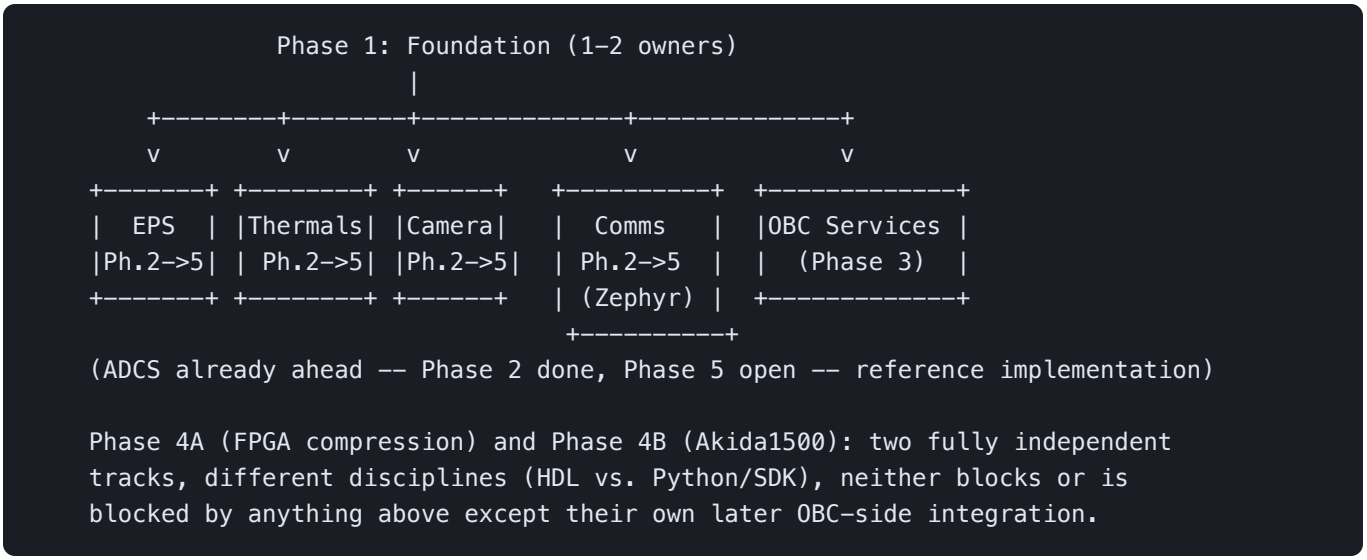
6 Hardware Bring-Up & System Integration

Depends on: at minimum one subsystem through Phase 5. Bench items — estimates are honest guesses and the least predictable in this document.

~11 tasks, ~33+ hours

#	Task	Depends on	Est.	Done when
6.1	Vendor STM32 HAL + real cross-compilation toolchain for the four FreeRTOS boards	Phase 5 (any one)	3h	Toolchain builds a real .elf for the target MCU
6.2	OBC: PetaLinux/Yocto build setup for the Zynq PS	Phase 5	3h (multi-session)	A bootable Linux image builds for the Zynq PS
6.3	Comms: real Zephyr build + flash to STM32H743 dev hardware	Phase 2B	3h	Comms boots and runs its Phase 2B scaffolding on real hardware
6.4	FPGA: Vivado bitstream build + PS-PL AXI bring-up on real board	Phase 4A	3h	4A.4's AXI round-trip test passes on real hardware
6.5	Akida1500: real hardware bring-up once the module is on hand	Phase 4B	3h	4B.4's integration milestone reproduces on real hardware
6.6	Fill in `platform/real/drivers/comms_i2c.c` against real I2C HAL/Linux driver headers	6.1, 6.2	3h	Real I2C traffic observed between two real boards
6.7	Per-subsystem bench bring-up: ADCS	6.6, ADCS Phase 5	3h	Phase 2's ADCS integration test passes on real hardware
6.8	Per-subsystem bench bring-up: EPS, Thermals, Camera (repeat 6.7's pattern per board)	6.6, each subsystem's Phase 5	3h each	Each subsystem's Phase 2 integration test passes on real hardware
6.9	Full end-to-end system test, including a real (or bridged) ground contact through Comms' radio path	6.3, 6.7, 6.8	3h	A ground-originated command reaches a subsystem and its telemetry returns, over real hardware

§ Suggested Parallelization Map



Six subsystem-shaped owners (EPS, Thermals, Camera, Comms, ADCS ahead, OBC Services) + two Phase 4 owners (FPGA/HDL, Akida/ML) + one foundation/infra owner. Up to nine people covers the whole graph without anyone blocking anyone else past Phase 1.

§ Definition of Done Checklists

Foundation piece (Phase 1 item)

- Documented in `directory_conventions.md` or a dedicated doc under `docs/`
- Has a working, minimal example — not just a header
- Used by at least one real consumer before being called "done"

New subsystem (Phase 2)

- Builds in `HW_MODE=OFF`
- One command flows end-to-end with a passing integration test
- Mock driver (`platform/sim/drivers/`) in place for anything hardware-facing
- Test was sabotage-verified once
- CSP address/ports match the table in this document

Comms follows the same four checks in spirit, via Zephyr's own build/emulation tooling instead of `HW_MODE`/`platform/sim/drivers/`.

Real logic replacing a mock (Phase 5)

- Mock backend still exists and still passes its own tests
- Real backend is either working against real hardware, or an honest stub matching the `platform/real/drivers/comms_i2c.c` pattern

FPGA compression (Phase 4A)

- Requirements written down (payload types, target ratio/latency) before any HDL work
- Software prototype validated the algorithm choice before HDL implementation
- Decompression format documented for the ground-segment owner

Akida1500 (Phase 4B)

- Requirements definition written down (input source, output consumer, success criteria) before any SDK/model work starts
- Physical interface confirmed against the actual module datasheet (Fork C resolved)
- First integration milestone is concretely defined, even if trivial

---

Flight Software V1 — Development Roadmap, revision 5. A living document: revisit as hardware constraints, team size, or mission requirements change the trade-offs.